

EAGER Project

Samira Malek, sxm6547@psu.edu

Under the supervision of Professor Sarah Rajtmajer

Rajtmajer Lab
Pennsylvania State University (PSU)



PennState

April 9, 2026

Table of Contents

1 Synthetic Prediction Market

2 Trainable Market Model

3 Multi-Class Market

4 LLM-agent Market Design

Agent Parameters:

For each agent $a_i \in \mathcal{A}$, for $0 \leq i \leq n$:

- Agent label $y_i \in \{0, 1\}$
- Reference center $h_i \in \mathbb{R}^d$
- Ellipsoid parameters $\theta_i \in \mathbb{R}_+^d$
- Scaling parameter $\alpha_i \in \mathbb{R}_+$
- Price sensitivity weight $w_p^i \in \mathbb{R}$
- Bias term $\theta_0^i \in \mathbb{R}$
- Initial budget $B_i \geq 0$

Synthetic Prediction Market

Initialization:

1: Market state: $q_0 = (q_0^0, q_0^1) = (0, 0)$

2: Initial prices:

$$p_0^k = \frac{e^{\beta q_0^k}}{e^{\beta q_0^0} + e^{\beta q_0^1}}, \quad k \in \{0, 1\}$$

3: Agent parameters $\forall a_i$:

- y_i, h_i, θ_i
- $\alpha_i, w_p^i, \theta_0^i$
- Budget $B_i > 0$

4: Input feature x

Market Dynamics:

1: **while** not converged **and** $t \leq T$ **do**

2: Activate agent a_i

3: Compute belief $\pi_t^i(x, p_t^{y_i})$

4: Compute cost $\kappa_t^{y_i}(1)$

5: **if** $\pi_t^i - \kappa_t^{y_i} \geq \tau$ **and** $B_i \geq \kappa_t^{y_i}(1)$ **then**

6: $q_t^{y_i} \leftarrow q_t^{y_i} + 1$

7: $B_i \leftarrow B_i - \kappa_t^{y_i}$

8: Update prices $p_t^k = \frac{\exp(\beta q_t^k)}{\exp(\beta q_t^0) + \exp(\beta q_t^1)}$

9: **end if**

10: **end while**

A Synthetic Prediction Market Model Trainable via Gradient Descent

- **Hyperparameters:**

- $N = 100$, 50% of y_i s are 0 and other 1.
- $\beta = 0.5$, $\tau = 0.1$
- $T = 1000$

- **Learning Parameters:**

- Reference center $h_i \in \mathbb{R}^d$
- Ellipsoid parameters $\theta_i \in \mathbb{R}^d$
- Scaling parameter $\alpha_i \in \mathbb{R}$
- Price sensitivity weight $w_p^i \in \mathbb{R}$
- Bias term $\theta_0^i \in \mathbb{R}$

- **Functions:**

- $\kappa^0(x) = \frac{1}{\beta} \log \left(\frac{\exp[\beta(q^0 + x)] + \exp[\beta q^1]}{\exp[\beta q^0] + \exp[\beta q^1]} \right)$
- $\kappa^1(x) = \frac{1}{\beta} \log \left(\frac{\exp[\beta(q^1 + x)] + \exp[\beta q^0]}{\exp[\beta q^0] + \exp[\beta q^1]} \right)$
- $\pi^i(x, p_t^{y_i}; h^i, \theta^i, \alpha^i, w_p^i) = \sigma \left[\alpha^i \left(1 - \sum_j \theta^{ij} (x - h_j^i)^2 \right) + w_p^i (p_t^{y_i} - p_0^{y_i}) + \theta_0^i \right]$

A Synthetic Prediction Market Model Trainable via Gradient Descent

Algorithm 1 Market Model

```
1: Initial:  $(q^0, q^1) = (0, 0), p_0^k = \frac{e^{\beta q^k}}{e^{\beta q^0} + e^{\beta q^1}}, k \in \{0, 1\}$ 
2: Input  $x$ 
3: for  $t = 1$  to  $T$  do
4:   for  $i = 0$  to  $N$  do
5:      $\pi^i = \pi^i(x, p_t^{y_i})$ 
6:      $\kappa^{y_i} = \kappa^{y_i}(1)$ 
7:     if  $\pi^i - \kappa^{y_i} \geq \tau$  then
8:        $q^{y_i} \leftarrow q^{y_i} + 1$ 
9:        $p_t^k = \frac{\exp(\beta q^k)}{\exp(\beta q^0) + \exp(\beta q^1)}, k \in \{0, 1\}$ 
10:    end if
11:  end for
12: end for
13: Output  $p_T^0, p_T^1$ 
```

A Synthetic Prediction Market Model Trainable via Gradient Descent

if-else decisions introduce non-differentiability, **preventing gradient-based** optimization.

- $\pi^i - \kappa^{y_i} \geq \tau \Rightarrow \pi^i - \kappa^{y_i} - \tau \geq 0$
- $q^{y_i} = q^{y_i} + \text{sign}(\text{ReLU}(\pi^i - \kappa^{y_i} - \tau))$

Smooth Surrogate Decision Function

Margin Definition

$$\text{margin}_{i,b} = \pi_{i,b} - \kappa_{y_i,b} - \tau$$

- $\pi_{i,b}$: agent belief (or predicted probability)
- $\kappa_{y_i,b}$: class-dependent cost
- τ : decision threshold

Hard Decision (Non-differentiable)

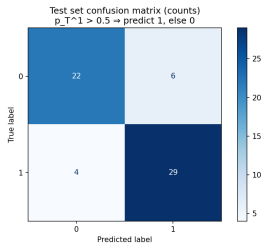
$$\text{step}_{i,b} = \begin{cases} 1 & \text{if } \text{margin}_{i,b} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Smooth Surrogate (Differentiable)

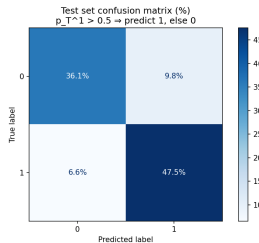
$$\text{step}_{i,b} = \sigma(s \cdot \text{margin}_{i,b}) = \frac{1}{1 + e^{-s \cdot \text{margin}_{i,b}}}$$

- $s = \text{soft_sharpness}$ controls steepness
- As $s \rightarrow \infty$, sigmoid $\rightarrow$ hard step function
- Enables gradient-based optimization

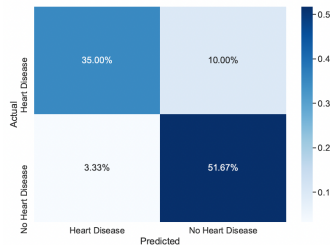
Results Overview for Heart Data



(a) confusion with number



(b) confusion with Percent



(c) Paper result

Figure 1: Result on Heart Data with 30 agents and $T = 20$

What Changes Are Needed from the Binary Model?

To generalize the binary market model to a three-class market, the following changes are required:

1. Replace the binary inventory vector

$$(q^0, q^1)$$

with the three-class inventory vector

$$(q^0, q^1, q^{-1}).$$

2. Replace the binary softmax probabilities with a three-class softmax:

$$p_t^k = \frac{\exp(\beta q^k)}{\exp(\beta q^0) + \exp(\beta q^1) + \exp(\beta q^{-1})}, \quad k \in \{0, 1, -1\}.$$

3. Extend the cost functions so that the update for each class depends on all three market states.

$$\kappa^k(x) = \frac{1}{\beta} \log \left(\frac{\exp[\beta(q^k + x)] + \sum_{j \neq k} \exp(\beta q^j)}{\exp(\beta q^0) + \exp(\beta q^1) + \exp(\beta q^{-1})} \right), \quad k \in \{0, 1, -1\}.$$

4. Replace the binary decision rule with the three-class prediction rule:

$$\hat{y} = \arg \max_{k \in \{0, 1, -1\}} p_t^k.$$

Three-Class Market Model

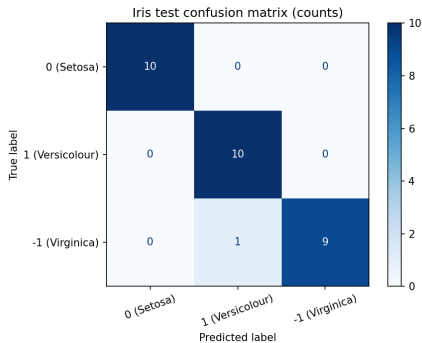
Algorithm 2 Three-Class Market Model

```

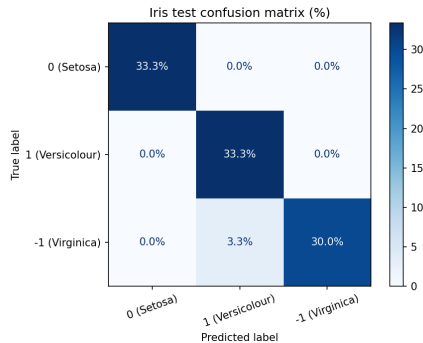
1: Initial:  $(q^0, q^1, q^{-1}) = (0, 0, 0)$ 
2:  $p_0^k = \frac{e^{\beta q^k}}{e^{\beta q^0} + e^{\beta q^1} + e^{\beta q^{-1}}}, \quad k \in \{0, 1, -1\}$ 
3: Input:  $x$ 
4: for  $t = 1$  to  $T$  do
5:   for  $i = 1$  to  $N$  do
6:      $\pi^i = \pi^i(x, p_t^{y_i})$ 
7:      $\kappa^{y_i} = \kappa^{y_i}(1)$ 
8:     if  $\pi^i - \kappa^{y_i} \geq \tau$  then
9:        $q^{y_i} \leftarrow q^{y_i} + 1$ 
10:     $p_t^k = \frac{\exp(\beta q^k)}{\exp(\beta q^0) + \exp(\beta q^1) + \exp(\beta q^{-1})}, \quad k \in \{0, 1, -1\}$ 
11:   end if
12: end for
13: end for
14: Output:  $p_T^0, p_T^1, p_T^{-1}$ 

```

Results Overview for iris Data



(a) confusion with number



(b) confusion with Percent

Figure 2: Result on Heart Data with 12 agents and $T = 300$. 40 data for training and 10 for testing for each class.

Initialization (Multi-Asset, Multi-Domain):

1: **Assets (popularity levels):** $\mathcal{K} = \{1, 2, 3, 4, 5\}$ (5 assets)

2: Market state: $q_0 = (q_0^1, \dots, q_0^5) = \mathbf{0}$

3: Initial prices (softmax/LMSR):

$$p_0^k = \frac{\exp(\beta q_0^k)}{\sum_{j \in \mathcal{K}} \exp(\beta q_0^j)}, \quad k \in \mathcal{K}$$

4: **Domain set (SMPD):** $\mathcal{D} = \{1, \dots, 11\}$ (e.g., Entertainment, Travel, Sport, Family, ...)

5: **LLM agents:** one expert per domain $\forall d \in \mathcal{D}$:

$$a_d \equiv \text{LLM expert in domain } d$$

6: Agent settings $\forall a_d$:

- Budget $B_d > 0$, position limit L_d
- Trading threshold τ_d
- (Optional) calibration temperature T_d

7: Input post x with domain label $d(x) \in \mathcal{D}$

8: (text + metadata from SMPD)

Market Dynamics (LLM Belief $\rightarrow$ Trade):

-
- 1: **while** not converged **and** $t \leq T$ **do**
 - 2: Select participants:

$$\mathcal{A}_t \leftarrow \{a_{d(x)}\} \cup \{a_{\text{gen}}\} \cup \text{Sample}(\mathcal{D} \setminus \{d(x)\})$$
 - 3: **for** each agent $a_d \in \mathcal{A}_t$ **do**
 - 4: **LLM belief over 5 assets:** $\pi_t^{(d)}(x) = (\pi_t^{(d),1}, \dots, \pi_t^{(d),5})$, $\sum_{k=1}^5 \pi_t^{(d),k} = 1$
 - 5: Compute LMSR cost of buying 1 share of asset k : $\kappa_t^k(1) = C(q_t + e_k) - C(q_t)$, $C(q) = \frac{1}{\beta} \log \left(\sum_{j=1}^5 \exp(\beta q^j) \right)$
 - 6: Choose best trade: $k^* \leftarrow \arg \max_{k \in \mathcal{K}} \left(\pi_t^{(d),k} - \kappa_t^k(1) \right)$
 - 7: **if** $\pi_t^{(d),k^*} - \kappa_t^{k^*}(1) \geq \tau_d$ **and** $B_d \geq \kappa_t^{k^*}(1)$ **and** $\text{pos}_d(k^*) < L_d$ **then**
 - 8: $q_t^{k^*} \leftarrow q_t^{k^*} + 1$
 - 9: $B_d \leftarrow B_d - \kappa_t^{k^*}(1)$
 - 10: Update prices: $p_t^k = \frac{\exp(\beta q_t^k)}{\sum_{j=1}^5 \exp(\beta q_t^j)}$, $k \in \mathcal{K}$
 - 11: **end if**
 - 12: **end for**
 - 13: **end while**
 - 14: **Output:** p_T (market consensus over 5 popularity levels), $\hat{k} = \arg \max_k p_T^k$
-

Initialization (Multi-Asset, Multi-Domain):

-
- 1: **Assets (popularity levels):** $\mathcal{K} = \{1, 2, 3, 4, 5\}$ (5 assets)
 - 2: Market state: $q_0 = (q_0^1, \dots, q_0^5) = \mathbf{0}$
 - 3: Initial prices (softmax/LMSR):

$$p_0^k = \frac{\exp(\beta q_0^k)}{\sum_{j \in \mathcal{K}} \exp(\beta q_0^j)}, \quad k \in \mathcal{K}$$

- 4: **Domain set (SMPD):** $\mathcal{D} = \{1, \dots, 11\}$ (e.g., Entertainment, Travel, Sport, Family, ...)
- 5: **LLM agents:** one expert per domain $\forall d \in \mathcal{D}$:

$$a_d \equiv \text{LLM expert in domain } d$$

- 6: Agent settings $\forall a_d$:
 - Trading threshold $\tau_d = 0.01$
 - (Optional) calibration temperature T_d
 - 7: Input post x with domain label $d(x) \in \mathcal{D}$
 - 8: (text + metadata from SMPD)
-

Market Dynamics (LLM Belief $\rightarrow$ Trade):

```

1: while not converged and  $t \leq T$  do
2:   Select participants:
        $\mathcal{A}_t \leftarrow \{a_{d(x)}\} \cup \{a_{\text{gen}}\} \cup \text{Sample}(\mathcal{D} \setminus \{d(x)\})$ 

3:   for each agent  $a_d \in \mathcal{A}_t$  do
4:     LLM belief over 5 assets:  $\pi_t^{(d)}(x) = (\pi_t^{(d),1}, \dots, \pi_t^{(d),5})$ ,  $\sum_{k=1}^5 \pi_t^{(d),k} = 1$ 
5:     Compute LMSR cost of buying 1 share of asset  $k$ :  $\kappa_t^k(1) = C(q_t + e_k) - C(q_t)$ ,  $C(q) =$ 
        $\frac{1}{\beta} \log \left( \sum_{j=1}^5 \exp(\beta q^j) \right)$ 
6:     Choose best trade:  $k^* \leftarrow \arg \max_{k \in \mathcal{K}} \left( \pi_t^{(d),k} - \kappa_t^k(1) \right)$ 
7:     if  $\pi_t^{(d),k^*} - \kappa_t^{k^*}(1) \geq \tau_d$  then
8:        $q^{k^*} \leftarrow q^{k^*} + 1$ 
9:       Update prices:  $p_t^k = \frac{\exp(\beta q^k)}{\sum_{j=1}^5 \exp(\beta q^j)}$ ,  $k \in \mathcal{K}$ 
10:    end if
11:  end for
12: end while
13: Output:  $p_T$  (market consensus over 5 popularity levels),  $\hat{k} = \arg \max_k p_T^k$ 

```

LLM Training Methods

- Zero-shot learning
- Few-shot learning

results

=== Market Parameters ===

- $\beta=1.0$, max-rounds=5, $\tau=0.01$
- n-sample-other-domains=3, seed=42

=== Final market state ===

- q (LMSR shares): [1.0, 1.0, 2.0, 1.0, 0.0]
- p (prices): [0.164307, 0.164307, 0.446633, 0.164307, 0.060445]
- $\hat{k}$ (argmax level 1..5): 3 , Ground Truth = 15.1

results

=== Last few trades (summary) ===

- domain-expert[Family]: $k^*=1$ trade=True max-edge=0.0403
- domain-expert[Whether&Season]: $k^*=2$ trade=False max-edge=0.0013
- domain-expert[Entertainment]: $k^*=2$ trade=False max-edge=0.0013
- domain-expert[Travel&Active&Sports]: $k^*=5$ trade=False max-edge=0.0012
- generalist: $k^*=5$ trade=False max-edge=0.0012
- domain-expert[Electronics]: $k^*=2$ trade=False max-edge=0.0013
- domain-expert[Urban]: $k^*=5$ trade=False max-edge=0.0012
- domain-expert[Whether&Season]: $k^*=5$ trade=False max-edge=0.0012

Thank You!